



Shri Vile Parle Kelavani Mandal's

DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Autonomous College Affiliated to the University of Mumbai)

NAAC Accredited with "A" Grade (CGPA : 3.18)



NAME : Abhishek Pandey

SAP ID: 60019220110

ROLL NO: B073

SUB: Network Security

NSL Experiment 6

Implementation of Encryption & Decryption using OpenSSL

1. Aim

To implement **data encryption and decryption using OpenSSL** and understand the working of **symmetric encryption (AES)** and **asymmetric encryption (RSA)**.

2. Introduction

During communication between systems, sensitive information such as passwords, messages, or confidential data may travel across networks. If this data is not protected, attackers may intercept and read it. To prevent this, **encryption** is used.

Encryption converts readable information (**plaintext**) into unreadable data (**ciphertext**) using cryptographic algorithms and keys. Only authorized users with the correct key can convert the ciphertext back to plaintext using **decryption**.

Two major encryption techniques are used in modern systems:

Symmetric Encryption

- Same key used for both encryption and decryption
- Faster and efficient for large data
- Example algorithm: **AES (Advanced Encryption Standard)**

Asymmetric Encryption

Uses two keys: a **Public key** and a **Private key**

- Public key encrypts data; Private key decrypts data
- Example algorithm: **RSA**

In this experiment we use **OpenSSL**, a powerful cryptographic toolkit widely used in secure systems such as HTTPS and SSH.



3. Prepare the Vagrant Environment

Extract the provided **ZIP file** and open the extracted folder in **VS Code**.

Inside the project folder you will find a **Vagrantfile**. Open it and replace its contents with the following configuration:

```
Vagrant.configure("2") do |config|

  config.vm.define "sender" do |sender|
    sender.vm.box = "ubuntu/focal64"
    sender.vm.hostname = "sender"
    sender.vm.network "private_network", ip: "192.168.56.20"
  end

  config.vm.define "receiver" do |receiver|
    receiver.vm.box = "ubuntu/focal64"
    receiver.vm.hostname = "receiver"
    receiver.vm.network "private_network", ip: "192.168.56.21"
  end

end
```

This configuration creates two virtual machines:

Machine	Role	IP Address
sender	encrypts data	192.168.56.20
receiver	decrypts data	192.168.56.21

These machines simulate communication between two systems on a private network.



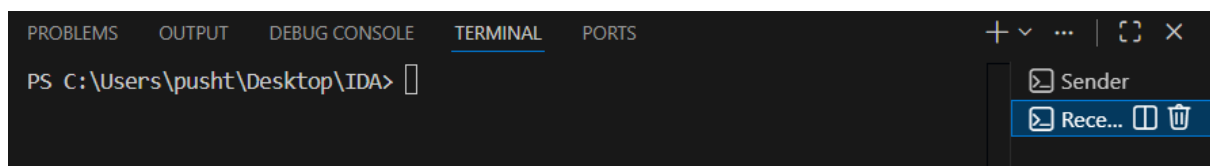
4. Start the Virtual Machines

```
PS C:\Users\pusht\Desktop\IDA> vagrant up
Bringing machine 'sender' up with 'virtualbox' provider...
Bringing machine 'receiver' up with 'virtualbox' provider...
```

```
sender: Guest Additions Version: 6.1.50
sender: VirtualBox Version: 7.2
==> sender: Setting hostname...
==> sender: Configuring and enabling network interfaces...
==> sender: Mounting shared folders...
sender: C:/Users/pusht/Desktop/IDA => /vagrant
receiver: Guest Additions Version: 6.1.50
receiver: VirtualBox Version: 7.2
==> receiver: Setting hostname...
==> receiver: Configuring and enabling network interfaces...
==> receiver: Mounting shared folders...
receiver: C:/Users/pusht/Desktop/IDA => /vagrant
PS C:\Users\pusht\Desktop\IDA> █
```

5. Create Two Terminals in VS Code

Open two terminals in VS Code and rename them:





Shri Vile Parle Kelavani Mandal's

DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Autonomous College Affiliated to the University of Mumbai)

NAAC Accredited with "A" Grade (CGPA : 3.18)



6. Login to the Virtual Machines

Terminal 1 — Login to Sender:

```
PS C:\Users\pusht\Desktop\IDA> vagrant ssh sender
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-216-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

System information as of Wed Mar 25 12:59:56 UTC 2026

System load:            0.05
Usage of /:              3.7% of 38.70GB
Memory usage:           21%
Swap usage:             0%
Processes:              117
Users logged in:        0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:1a:a2ff:fe1e:9db7

Expanded Security Maintenance for Infrastructure is not enabled.

0 updates can be applied immediately.

Enable ESM Infra to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '22.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

vagrant@sender:~$
```



Shri Vile Parle Kelavani Mandal's

DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Autonomous College Affiliated to the University of Mumbai)

NAAC Accredited with "A" Grade (CGPA : 3.18)



Terminal 2 — Login to Receiver:

```
PS C:\Users\pusht\Desktop\IDA> vagrant ssh receiver
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-216-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Mar 25 13:01:36 UTC 2026

System load:          0.08
Usage of /:           3.7% of 38.70GB
Memory usage:        21%
Swap usage:           0%
Processes:            117
Users logged in:      0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:1a:a2ff:fe1e:9db7

Expanded Security Maintenance for Infrastructure is not enabled.

0 updates can be applied immediately.

Enable ESM Infra to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '22.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

vagrant@receiver:~$
```

7. Install OpenSSL

On Sender (Terminal 1)

Check if OpenSSL is already installed:

```
vagrant@sender:~$ openssl version
OpenSSL 1.1.1f  31 Mar 2020
vagrant@sender:~$
```

If not installed, run:

```
sudo apt update && sudo apt install openssl -y
```



Shri Vile Parle Kelavani Mandal's

DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Autonomous College Affiliated to the University of Mumbai)

NAAC Accredited with "A" Grade (CGPA : 3.18)



On Receiver (Terminal 2)

```
vagrant@receiver:~$ openssl version
OpenSSL 1.1.1f 31 Mar 2020
vagrant@receiver:~$
```

If not installed:

```
sudo apt update && sudo apt install openssl -y
```

PART A — Symmetric Encryption using AES

Symmetric encryption uses **the same password for both encryption and decryption**. We use **AES-256-CBC**, one of the strongest symmetric encryption standards.

****🔒 Strong Password for this experiment:**T#8rVx!2mQpL\$9wZ** This password will be used to ****encrypt**** the file on the Sender and ****decrypt**** it on the Receiver. Keep it safe — without it, the file cannot be decrypted.

Step 1 — Create a Plaintext Message (Sender)

```
vagrant@sender:~$ nano message.txt
vagrant@sender:~$
```

```
GNU nano 4.8 message.txt
CONFIDENTIAL BANK TRANSFER NOTICE
Date: 2025-06-15
From Account: SB-00284-ALPHA
To Account: SB-99103-DELTA
Amount: USD 47,500.00
Authorization Code: BT-2025-XK9

This transfer has been approved by the finance department.
Do not share this message with unauthorized personnel.
```



Step 2 — Encrypt the File (Sender)

```
vagrant@sender:~$ openssl enc -aes-256-cbc -salt -pbkdf2 -in message.txt -  
out encrypted_aes.bin  
enter aes-256-cbc encryption password:  
Verifying - enter aes-256-cbc encryption password:  
vagrant@sender:~$
```

When prompted, enter the password:

```
T#8rVx!2mQpL$9wZ
```

What OpenSSL does here:

1. Reads the plaintext file
2. Derives a 256-bit encryption key from the password using PBKDF2
3. Applies AES-256-CBC encryption with a random salt
4. Produces ciphertext output

Now view the encrypted file to confirm it is unreadable:

```
vagrant@sender:~$ cat encrypted_aes.bin  
Salted__g7{?T?LUöi?A?  
?h?0?~?6?i?~?>?d?P?bP  
?z?lce?WA?j\穿?s?!?f?c?v|>?^,?=nf?v?5ç@?j?z?F?2l?Gl?  
{C?ö?N ?m?x?93  
oz?7?M}u?z?C):?7Y?8?L?  
'?X?1d|v ?HI?  
?7?I?SX?s?-?n  
???z???*??:)*S??Np??B??4??6=?T?yB-?  
??P?&z??<2{w?l?m=?g?vagrant@sender:~$
```

You will see output as — garbled, unreadable binary data

The plaintext is now fully protected. Even though this is the same file you created, its contents are completely unrecognizable.



Step 3 — Transfer Encrypted File via Shared Folder (Sender)

```
vagrant@sender:~$ cp encrypted_aes.bin /vagrant/
vagrant@sender:~$
```

****i About `/vagrant`:**** This directory is automatically shared between both virtual machines by Vagrant. Any file placed here on one machine is instantly accessible on the other.

Step 4 — Retrieve Encrypted File (Receiver)

```
vagrant@receiver:~$ cp /vagrant/encrypted_aes.bin .
vagrant@receiver:~$
```

Before decrypting, verify that the received file is still encrypted and unreadable:

```
vagrant@receiver:~$ cat encrypted_aes.bin
Salted__g7{?T?LUöi?A?
?h?0?6?i?~?>?d?P?bP
?z?lce?WA?j\穿?s?!?f?c?v?|?>?^,=?nf? v?5?@?j?z?F?2l?G?l?%
{C?ö?N ?m?x?93
oz?7?M?u?z?C):?7Y?8?L?
'X?1d?v ?HI?
?7?I?SX?s?-?n
??z???'*??:)*S?Np?B?4?6=?T?yB-
??P?&z??<2{w?l?m=?g?vagrant@receiver:~$
```

You will see the same garbled ciphertext as on the Sender — confirming the file transferred correctly and has not been tampered with. The original message is still hidden inside.

Step 5 — Decrypt the File (Receiver)

```
??P?&z??<2{w?l?m=?g?vagrant@receiver:~$ openssl enc -aes-256-cbc -d -pbkdf2 -in encrypted_aes.bin -out decrypted_aes.txt
enter aes-256-cbc decryption password:
vagrant@receiver:~$
```

When prompted, enter the **same password used during encryption**:

```
T#8rVx!2mQpL$9wZ
```



Step 6 — View the Decrypted Message (Receiver)

```
vagrant@receiver:~$ cat decrypted_aes.txt
CONFIDENTIAL BANK TRANSFER NOTICE
Date: 2025-06-15
From Account: SB-00284-ALPHA
To Account:   SB-99103-DELTA
Amount:      USD 47,500.00
Authorization Code: BT-2025-XK9

This transfer has been approved by the finance department.
Do not share this message with unauthorized personnel.
vagrant@receiver:~$
```

The ciphertext has been successfully converted back to the original plaintext.

PART B — Asymmetric Encryption using RSA

Asymmetric encryption uses a **key pair**:

The **Public Key** encrypts data — it can be shared freely

- The **Private Key** decrypts data — it must never be shared

The receiver generates both keys. The sender uses the receiver's public key to encrypt, and only the receiver's private key can decrypt it.

Step 1 — Generate RSA Private Key (Receiver)

```
vagrant@receiver:~$ openssl genrsa -out private.pem 2048
Generating RSA private key, 2048 bit long modulus (2 primes)
.....+++++
.....+++++
e is 65537 (0x010001)
vagrant@receiver:~$
```

This generates a **2048-bit RSA private key**. This file must remain on the receiver only and must never be shared.



Step 2 — Extract the Public Key (Receiver)

```
vagrant@receiver:~$ openssl rsa -in private.pem -pubout -out public.pem
writing RSA key
vagrant@receiver:~$
```

This derives the public key from the private key. The public key can be shared safely with anyone who wishes to send you an encrypted message.

Step 3 — Share the Public Key via Shared Folder (Receiver)

```
vagrant@receiver:~$ cp public.pem /vagrant/
vagrant@receiver:~$
```

Step 4 — Retrieve the Public Key (Sender)

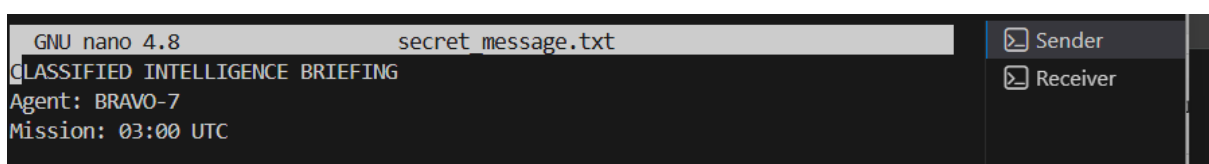
```
vagrant@sender:~$ cp /vagrant/public.pem .
vagrant@sender:~$
```

The sender now has the receiver's public key and can use it to encrypt a message.

Step 5 — Create a New Plaintext Message (Sender)

```
vagrant@sender:~$ nano secret_message.txt
```

Type the following message:



Save and close the file.

Step 6 — Encrypt the Message using the Public Key (Sender)

```
vagrant@sender:~$ openssl pkeyutl -encrypt -inkey public.pem -pubin -in secret_message.txt -out rsa_encrypted.bin
```

What OpenSSL does here:

1. Reads the receiver's public key
2. Encrypts the message using the RSA algorithm



3. Produces ciphertext that only the receiver's private key can unlock

Now inspect the encrypted file on the Sender to confirm it is unreadable:

```
vagrant@sender:~$ cat encrypted_aes.bin
Salted__g70{T0?00LU00i0 0A000
00h0000000`0600i0~00>00d000000$0P0bb
P0z0lce000WA0j\穿0s000!00f00c000v0|0>000^,0=nf00 v0005c@0j0z00F02l0000G10%
{C0000N 0m00x093
oz0700M}u0z0C):007Y0800L0
'0X01d|v 0HI00
0700I0SX0s00-0n
000z000'0*000000:0)S00Np00B004006=0T0yB-0
00P0&z0?0<2{w0l0m=00g00vagrant@sender:~$
```

You will see output like this — completely scrambled binary data with no resemblance to the original message. This confirms the RSA encryption is working — the 256-byte RSA-encrypted block is unreadable without the private key.

Step 7 — Transfer RSA Encrypted File via Shared Folder (Sender)

```
vagrant@sender:~$ cp rsa_encrypted.bin /vagrant/
```

Step 8 — Retrieve the Encrypted File (Receiver)

```
vagrant@receiver:~$ cp /vagrant/rsa_encrypted.bin .
vagrant@receiver:~$
```

Before decrypting, verify the file is still encrypted and unreadable on the Receiver as well:

```
vagrant@receiver:~$ cat rsa_encrypted.bin
00i^N00X02;\0%
%00000&28_"0w`005w0tp'00g0Wg0&00o0*00^0%00<0*000p0<z000a0R00000
!b0000000|0k-0Ig'0}0=0p0g|0yp00%h~0C00t00h00U{[ow0AlI?2>3p/0\0i0
0¥0$0+`0@0]gj00000J*0p00a!'i030008DAA00)00Ä0wK20!Y?00|0f000000Vvagrant@recei
ver:~$
```

The output should be the same garbled ciphertext — confirming the file arrived intact. Without the private key (private.pem), this data is completely indecipherable.

Step 9 — Decrypt the File using the Private Key (Receiver)

```
vagrant@receiver:~$ openssl pkeyutl -decrypt -inkey private.pem -in rsa_en
crypt.ed.bin -out rsa_decrypted.txt
```



Step 10 — View the Decrypted Message (Receiver)

```
vagrant@receiver:~$ cat rsa_decrypted.txt
CLASSIFIED INTELLIGENCE BRIEFING
Agent: BRAVO-7
Mission: 03:00 UTC
vagrant@receiver:~$
```

The original message has been successfully recovered using only the private key.

8. Result

The experiment successfully demonstrated:

- **AES-256-CBC symmetric encryption** using a strong shared password
- **Secure file transfer** between virtual machines using the shared /vagrant folder
- **RSA asymmetric encryption** using a 2048-bit public/private key pair
- Correct decryption of both AES and RSA encrypted files

9. Conclusion

In this experiment we implemented symmetric and asymmetric encryption using OpenSSL across two virtual machines. AES encryption used a shared strong password (T#8rVx!2mQpL\$9wZ) for both encryption and decryption, demonstrating speed and simplicity for bulk data. RSA encryption used a public/private key pair, ensuring that only the intended recipient holding the private key could decrypt the message — even if the encrypted file was intercepted in transit.

These encryption techniques form the foundation of modern secure communication systems including HTTPS, SSH, PGP email encryption, and end-to-end encrypted messaging platforms.